

习题 3

3.1 简述硬件描述语言的定义及其作用，并举例说明。

答：硬件描述语言（HDL）是对电子系统的硬件进行行为描述、结构描述和数据流描述的语言，它以文本形式来描述数字系统的硬件结构和行为，包括逻辑电路图和逻辑表达式，以及数字逻辑系统所完成的逻辑功能。其主要作用包括电路设计、仿真验证、综合优化等。常用的硬件描述语言包括 Verilog HDL、VHDL、SystemVerilog 等。

3.2 简述 Verilog HDL 和 VHDL 之间的区别，并举例说明。

答：VHDL 是强类型语言；VHDL 不区分大小写（连保留字也不区分大小写），Verilog HDL 则区分大小写（大小写含义不同）；VHDL 具有很多不同的复杂数据类型，Verilog HDL 语言的主要数据类型就简单许多，这使得 Verilog HDL 对系统级建模的能力较弱，但新一代的 Verilog HDL 语言，如 Verilog-2001 及 SystemVerilog 等，就针对系统级的部分进行了加强，且完全向下兼容；Verilog HDL 语言因其可程序化的接口，可以无限扩充而成为功能强大的硬件设计语言。VHDL 在这方面有所欠缺，VHDL 语言以 Package 的形式取代。

举例：（1）Verilog 的语法类似 C 语言，简洁直观，适合有软件背景的开发者的。

```
module adder(  
    input [3:0] a, b,  
    output [4:0] sum);  
    assign sum = a + b;  
endmodule
```

（2）VHDL 的语法类似 Ada，强调强类型和结构化设计，代码更冗长但严谨。

```
entity adder is  
    port (  
        a, b : in  std_logic_vector(3 downto 0);  
        sum : out std_logic_vector(4 downto 0)  
    );  
end entity;  
  
architecture Behavioral of adder is  
begin  
    sum <= ('0' & a) + ('0' & b); -- 显式位宽扩展
```

```
end architecture;
```

3.3 简述 Verilog HDL 和 C 之间的区别，并举例说明。

答：Verilog HDL 是一种硬件描述语言，其作用是进行电路设计，描述电路的功能、连接和时序。C 语言是一种软件描述语言，其作用是通过算法逻辑实现某个功能，但不涉及电路如何实现此功能等。C 语言经过编译后，最终转化为处理器能够执行的二进制目标代码；Verilog HDL 描述的是硬件电路实现，其最大的优点就是：硬件电路可以并行执行。因此，Verilog HDL 的部分描述语句可以并行执行，与语句出现的顺序无关。C 语言则只能串行执行，在上一条语句执行结束之后，才能执行下一条语句，代码的顺序决定了执行顺序；Verilog HDL 有着比 C 语言更加丰富的抽象层次，更大程度上提高电路性能和灵活性。

3.4 SystemVerilog 是 Verilog HDL 的扩展。请研究 SystemVerilog 和 Verilog HDL 的历史，给出这两种语言的最新标准，总结 Verilog HDL 中没有而 SystemVerilog 中具有的新特性。

答：Verilog HDL 起源于 1984 年，由 Gateway Design Automation 公司开发，最初用于逻辑仿真和综合。1995 年成为 IEEE 标准 IEEE 1364-1995 (Verilog-1995)；2001 年扩展为 IEEE 1364-2001 (Verilog-2001)，引入多维数组、生成块等功能；2005 年最后一次独立更新为 IEEE 1364-2005 (Verilog-2005)，此后不再单独发展。

SystemVerilog 起源于 2002 年，由 Accellera (电子设计自动化行业联盟) 推出，结合了 Verilog 和 SuperLog 语言的特性。2005 年成为首个 IEEE 标准 IEEE 1800-2005，明确 SystemVerilog 为 Verilog 的超集；2009 年与 IEEE 1364 (Verilog) 合并为统一标准 IEEE 1800-2009。2012/2017 年更新为 IEEE 1800-2012 和 IEEE 1800-2017 (当前最新版)，增强验证和设计功能。

systemVerilog 在 Verilog 基础上扩展了设计增强和验证增强两大方向的功能。

3.5 Verilog HDL 包括哪些基本语言要素？

答：Verilog HDL 的核心语言要素包括：

- (1) 模块化设计 (module、端口申明、内部逻辑、endmodule)。
- (2) 数据类型 (wire、reg、integer)。
- (3) 运算符与表达式 (算术运算符、位运算符、逻辑运算符、关系运算符、移位运算符、拼接运算符)。
- (4) 过程块 (always、initial)。
- (5) 控制结构 (条件语句、循环语句)
- (6) 任务与函数 (代码复用)。
- (7) 层次化设计 (实例化、构建复杂系统)。
- (8) 编译指令与系统任务 (控制仿真环境)。

(9) 测试平台 (Testbench)

3.6 Verilog HDL 包括哪两种注释方式?

答: Verilog HDL 中有两种注释的方式:

(1) 单行注释符: 以“//”符号开头的语句, 它表示从“//”开始到本行结束都属于注释语句。

(2) 多行注释符 (用于编写多行注释): 以“/*”符号开始, “*/”符号结束, 在两个符号之间的语句都是注释语句, 因此可扩展到多行。多行注释不允许嵌套, 但单行注释可以嵌套在多行注释中。

3.7 下列哪些是合法的标识符?

- (1) begin (2) _input (3) 3abc (4) \out
(5) data_1 (6) @123 (7) din\$ (8) Rst_n

答: (2)、(5)、(7)、(8)

3.8 简要分析 wire 型和 reg 型数据的区别?

答: wire 是线网类型, 表示 Verilog HDL 结构化元件间的物理连线, 它的值由驱动元件的值决定。reg 是寄存器类型, 表示一个抽象的数据存储单元, 它只能在 always 语句和 initial 语句中被赋值, 并且它的值能够从一个赋值到另一个赋值过程中被保存下来。

3.9 Verilog HDL 中定义存储器类型数据的方法是?

答: reg[n-1:0] 存储器名[m-1:0]; //定义 1 个位宽为 n, 深度为 m 的存储器型变量

3.10 简述运算符“~”和“!”的区别, 并举例说明。

答: “~”是按位运算符, 表示将 1 个多位操作数按位取反。“!”是逻辑运算符, 表示对 1 个操作数进行逻辑取反, 输出结果是一位逻辑值。

3.11 简述运算符“&&”和“&”的区别, 并举例说明。

答: “&&”是逻辑运算符, 表示对 2 个操作数进行逻辑与, 输出结果是一位逻辑值。“&”是按位运算符, 表示将 2 个多位操作数按位进行与运算, 各位运算的结果按顺序组成一个新的多位数。

3.12 简述右移和算术右移的区别, 并举例说明。

答: 右移的运算符是“>>”, 表示对 1 个操作数进行右移操作, 左边空出的高位补 0。算术右移的运算符是“>>>”, 表示对 1 个操作数进行算术右移操作, 左边空出高位会补充符号位 (符号位为 1 就补 1, 符号位为 0 就补 0)。

3.13 简述逻辑相等“==”和 case 相等“===”的区别, 并举例说明。

答: 逻辑相等“==”表示 2 个操作数比较, 如果各 bit 均相等, 则结果为真, 否则为假。如果其中任何一个操作数中含有 X 或 Z, 则结果为 X。case 相等“===”表示 2 个操作数比较, 如果各 bit (包括 X 和 Z) 均相等, 则结果为真, 否则为假。

3.14 Verilog HDL 模块由哪些部分构成? 每个部分由什么语句构成?

答：Verilog HDL 模块一般由模块声明、端口定义、内部信号声明、逻辑功能描述等构成。模块声明语句：`module` 和 `endmodule`。端口声明语句：`input`、`output`、`inout`。内部声明语句：`wire`、`reg`、`parameter` 等。功能描述语句：组合逻辑 `assign`、`always @(*)`、时序逻辑 `always @(posedge clk)`。子模块实例化语句：模块名 + 实例名 + 端口连接。可选部分语句：参数化、生成块、任务/函数。

3.15 Verilog HDL 模块的端口包括哪几种？模块的端口可以采用什么方式进行描述？

答：Verilog HDL 模块的端口包括：输入（`input`）信号端口、输出（`output`）信号端口和双向（`inout`）信号端口。

`input`：模块从外界读取数据的接口，在模块内不可写；

`output`：模块往外界发送数据的接口，在模块内不可读；

`inout`：可读取数据，也可发送数据，数据可双向流动；

3.16 Verilog HDL 模块实例化的方式是什么？请举例说明。

答：Verilog HDL 模块实例化语句的格式为：

模块名<参数值列表> 实例名(端口列表);

`fulladder u_fulladder (A, B, Cin, S, Cout);`

`fulladder u_fulladder (.a (A), .b (B), .cin(Cin), .s(S), .cout(Cout));`

3.17 Verilog HDL 模块实例化时端口列表可以采用哪两种匹配方式？比较两种方式的优缺点。

答：端口位置匹配方式和端口信号名匹配方式。

端口位置匹配方式严格按照模块定义的端口顺序与外部信号进行匹配连接，位置要严格保持一致，但不用标明原模块定义时规定的端口名。

端口信号名匹配方式将被实例化的模块端口与外部信号按照模块定义时的端口名进行连接，而不是按照位置。采用“.”符号，端口连接可以以任意顺序出现，只要保证端口名与外部信号匹配即可，但需标明被实例化模块定义时规定的端口名。

3.18 简述连续赋值语句和过程赋值语句的区别，并举例说明。

答：连续赋值语句是在 `initial` 或 `always` 块语句以外的赋值语句。它是 Verilog HDL 数据流建模的基本语句，主要对线网数据类型变量进行赋值，等价于门级描述，一般在描述纯组合逻辑电路时使用。

过程赋值语句是在 `initial` 或 `always` 块语句以内的赋值语句。它主要对寄存器数据类型的变量赋值，变量在被赋值后，其值将保持不变，直到被其他过程赋值语句赋予新值。过程赋值语句只有在执行到的时候才会起作用，它只能在 `initial` 或 `always` 块语句内进行赋值，且 `initial` 和 `always` 块中也只能使用过程赋值语句。

3.19 简述 `always` 过程块和 `initial` 过程块的区别，并举例说明。

答：`initial` 语句是从 0 时刻开始执行，只执行一次，多个 `initial` 块之间是相互独立的。`always` 语句

则是重复执行的。`always` 语句块从 0 时刻开始执行其中的行为语句；当执行完最后一条语句后，便判断敏感信号列表内的事件是否发生。如果敏感事件发生则会再次执行语句块中的语句，如此循环反复。

3.20 简述阻塞赋值语句和非阻塞赋值语句的区别，并举例说明。

答：阻塞赋值采用“`=`”运算符，当一个块语句中包含若干条阻塞过程赋值语句时，这些赋值语句是按照语句编写的顺序由上至下一条一条地执行。如果上一条语句没有完成，则下一条语句就无法执行，像被“阻塞”了一样。阻塞赋值语句常用于描述组合逻辑电路。

非阻塞赋值采用“`<=`”运算符，与阻塞赋值不同，一条非阻塞赋值语句的执行是不会阻塞下一条语句的执行，也就是说在本条非阻塞赋值语句执行完毕前，下一条语句也可开始执行。非阻塞赋值语句常用于描述时序逻辑电路。

3.21 简述 `case`、`casex`、`casez` 之间的相同点和不同点，并举例说明。

答：在 Verilog HDL 中，`case`、`casex` 和 `casez` 是用于多条件分支选择的关键字，它们的相同的包括（1）三者语法结构相同，均基于 `case` 语句框架。（2）执行逻辑相同，都是按顺序匹配第一个满足条件的分支，未匹配时执行 `default`。（3）应用场景相同，均用于实现多路选择逻辑（如状态机、解码器）。

它们的核心区别在于匹配规则和通配符处理，主要包括：（1）在 `case` 语句中，敏感表达式中与各项值之间是精确匹配（所有位必须相等），适用于严格匹配的精确逻辑。（2）在 `casez` 语句中，忽略高阻态 `z` 和通配符 `?`，而只关注其他位的比较结果，适用于允许部分位无关（如中断屏蔽）。（3）在 `casex` 语句中，忽略 `x`（未知）、`z` 和 `?`，这些位的比较不予考虑，适用于仿真调试或模糊匹配。

3.22 简述 Verilog HDL 的主要建模方式及区别。

答：Verilog HDL 的主要建模方式包括门级建模、数据流建模、行为建模、混合建模、状态机建模。门级建模通常描述一个小规模逻辑电路。数据流建模的抽象级别介于门级和行为级之间，它是行为级建模的一种形式，它可以让设计者根据数据流来优化电路，而不必专注于电路结构的细节，组合电路的逻辑行为最方便使用数据流建模方式。行为级建模是以抽象的形式描述数字逻辑电路的行为（功能）和算法。混合建模用于描述一个复杂的逻辑功能。状态机建模在控制复杂流程的建模中优势非常明显。

3.23 简述有限状态机的类型及区别。

答：有限状态机分为米利(Mealy)型状态机和摩尔(Moore)型状态机两大类。摩尔状态机和米利状态机的区别在于米利状态机的输出由当前状态和输入决定，而摩尔状态机的输出只取决于当前状态。

3.24 状态机常用状态编码有_____、_____和_____。

答：二进制码、格雷码、独热码。

3.25 Verilog HDL 中任务可以调用_____和_____。

答：其他任务和函数

3.26 系统函数和任务函数的首字符标志为_____，预编译指令首字符标志为_____。

答：\$、`

3.27 利用 Verilog HDL 门级建模方式实现逻辑函数 $f(X,Y,Z) = \sum m(0,3,4,5,7)$ ，要求完成设计源文件和仿真源文件的编制，并进行仿真分析。

答： $f(X,Y,Z) = \bar{X}\bar{Y}\bar{Z} + \bar{X}YZ + X\bar{Y}\bar{Z} + X\bar{Y}Z + XYZ$

门级建模方式 Verilog 代码：

```
module exercises3_27(  
    input X,  
    input Y,  
    input Z,  
    output F  
);  
wire N_X, N_Y, N_Z, R1, R2, R3, R4, R5;  
not  
    N1(N_X, X),  
    N2(N_Y, Y),  
    N3(N_Z, Z);  
and  
    A1(R1, N_X, N_Y, N_Z),  
    A2(R2, N_X, Y, Z),  
    A3(R3, X, N_Y, N_Z),  
    A4(R4, X, N_Y, Z),  
    A5(R5, X, Y, Z);  
or  
    O1(F, R1, R2, R3, R4, R5);  
endmodule
```

仿真代码：

```
`timescale 1ns / 1ps  
module tb_exercises3_27( );  
    reg X;  
    reg Y;
```

```

reg Z;

wire F;

initial begin
    X = 1'b0;
    Y = 1'b0;
    Z = 1'b0;
end

always begin
    #10
    {X,Y,Z} = {X,Y,Z} + 1'b1;
end

exercises3_27 uut(
    .X(X),
    .Y(Y),
    .Z(Z),
    .F(F));

endmodule

```

3.28 利用 Verilog HDL 数据流建模方式实现逻辑函数 $f(X,Y,Z) = \sum m(0,3,4,5,7)$ ，要求完成设计源文件和仿真源文件的编制，并进行仿真分析。

答：数据流建模方式 Verilog 代码：

```

module exercises3_28(
    input X,
    input Y,
    input Z,
    output F
);
    assign F = (~X & ~Y & ~Z) | (~X & Y & Z) | (X & ~Y & ~Z) | (X & ~Y & Z) | (X & Y & Z);
endmodule

```

仿真代码如题 3.27。

3.29 利用 Verilog HDL 行为建模方式实现逻辑函数 $f(X,Y,Z) = \sum m(0,3,4,5,7)$ ，要求完成设计源文件和仿真源文件的编制，并进行仿真验证。

答：真值表如下：

X	Y	Z	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

行为建模方式 Verilog 代码：

```
module exercises3_29(  
    input X,  
    input Y,  
    input Z,  
    output reg F);  
    always @(X or Y or Z) begin  
        case({X,Y,Z})  
            3'b000: F = 1'b1;  
            3'b001: F = 1'b0;  
            3'b010: F = 1'b0;  
            3'b011: F = 1'b1;  
            3'b100: F = 1'b1;  
            3'b101: F = 1'b1;  
            3'b110: F = 1'b0;  
            3'b111: F = 1'b1;  
        endcase  
    end  
endmodule
```


仿真代码如题 3.27。

3.30 一条长长的走廊有三扇门，两头各一扇，中间一扇。每扇门都有一个开关控制走廊上的灯，将开关标记为 A、B 和 C。假设灯是关着的，则拨动任何一个开关，灯都会打开。或者，如果灯是开着的，则拨动任何一个开关，灯都会关闭。请编写控制灯的 Verilog HDL 设计源文件和仿真源文件，并进行仿真验证。

答：三个开关（A/B/C）控制灯的亮灭，任意开关拨动翻转灯状态，Verilog 代码如下：

```
module exercises3_30(
    input  clk,      // 时钟信号（用于同步检测开关变化）
    input  rst,
    input  A,        // 开关 A
    input  B,        // 开关 B
    input  C,        // 开关 C
    output reg light  // 灯状态（0=关，1=开）
);

// 保存开关前一时刻状态的寄存器
reg prev_A, prev_B, prev_C;

always @(posedge clk or negedge rst) begin
    if(!rst) begin
        prev_A <= 1'b0;
        prev_B <= 1'b0;
        prev_C <= 1'b0;
        light  <= 1'b0;
    end
    else begin
        // 检测任一开关状态变化（上升沿或下降沿）
        if((A != prev_A) || (B != prev_B) || (C != prev_C)) begin
            light <= ~light; // 翻转灯状态
        end

        // 更新开关状态记录
        prev_A <= A;
        prev_B <= B;
        prev_C <= C;
    end
end
```

```
endmodule
```

仿真文件如下：

```
`timescale 1ns/1ps // 定义仿真时间单位和精度
module tb_exercises3_30 ();
// 定义信号
reg clk;
reg rst;
reg A, B, C;
wire light;

// 实例化被测测试模块
exercises3_30 uut (
    .clk(clk),
    .rst(rst),
    .A(A),
    .B(B),
    .C(C),
    .light(light)
);

// 生成时钟信号（周期=10ns）
initial begin
    clk = 0;
    rst = 0;
    #10
    rst = 1;
    forever #5 clk = ~clk; // 每 5ns 翻转一次，周期 10ns
end

// 测试逻辑
initial begin
    // 初始化信号
    A = 0;
    B = 0;
    C = 0;
    #15; // 等待时钟稳定和初始状态记录

    // 测试用例 1：拨动开关 A（灯应打开）
```

```

    A = 1;
    #10; // 等待时钟边沿检测
    A = 0;
    #10;

    // 测试用例 2: 拨动开关 B (灯应关闭)
    B = 1;
    #10;
    B = 0;
    #10;

    // 测试用例 3: 拨动开关 C (灯应打开)
    C = 1;
    #10;
    C = 0;
    #10;

    // 测试用例 4: 同时拨动 A 和 B (灯应关闭)
    A = 1;
    B = 1;
    #10;
    A = 0;
    B = 0;
    #10;
end
endmodule

```

3.31 设计一个 3 位二进制数比较器，输入为 $A=a_2a_1a_0$ 和 $B=b_2b_1b_0$ ，三个输出分别为 $EQ(A=B)$ 、 $GT(A>B)$ 和 $LT(A<B)$ 。请编写控制灯的 Verilog HDL 设计源文件和仿真源文件，并进行仿真验证。

答：3 位二进制数比较器，输出 EQ （相等）、 GT （大于）、 LT （小于），Verilog 代码如下：

```

module exercises3_31 (
    input  [2:0] A, // 输入 A = a2a1a0
    input  [2:0] B, // 输入 B = b2b1b0
    output EQ,      // A == B
    output GT,      // A > B
    output LT       // A < B
);

```

```

// EQ 逻辑：所有位相等
assign EQ = (A == B);

// GT 逻辑：逐位比较，从高位到低位
assign GT = (A[2] > B[2]) ||
            ((A[2] == B[2]) && (A[1] > B[1])) ||
            ((A[2] == B[2]) && (A[1] == B[1]) && (A[0] > B[0]));

// LT 逻辑：逐位比较，从高位到低位
assign LT = (A[2] < B[2]) ||
            ((A[2] == B[2]) && (A[1] < B[1])) ||
            ((A[2] == B[2]) && (A[1] == B[1]) && (A[0] < B[0]));

endmodule

```

仿真代码如下：

```

`timescale 1ns/1ps
module tb_exercises3_31();
// 定义信号
reg [2:0] A, B;
wire EQ, GT, LT;

// 实例化被测测试模块
exercises3_31 uut (
    .A(A),
    .B(B),
    .EQ(EQ),
    .GT(GT),
    .LT(LT)
);
// 测试逻辑
initial begin
    // 初始化输入
    A = 3'b000;

```

```
B = 3'b000;

#10;

// 测试用例 1: A == B

A = 3'b101; B = 3'b101;

#10;

// 测试用例 2: A > B (高位不同)

A = 3'b111; B = 3'b011;

#10;

// 测试用例 3: A > B (高位相同, 低位不同)

A = 3'b110; B = 3'b100;

#10;

// 测试用例 4: A > B (高位和次高位相同, 最低位不同)

A = 3'b101; B = 3'b100;

#10;

// 测试用例 5: A < B (高位不同)

A = 3'b001; B = 3'b010;

#10;

// 测试用例 6: A < B (高位相同, 低位不同)

A = 3'b010; B = 3'b011;

#10;

// 测试用例 7: 混合情况

A = 3'b000; B = 3'b111;

#10;

end

endmodule
```

3.32 利用 Verilog HDL 行为建模方式设计一个逻辑电路, 将 4 位二进制数字从有符号格式转换为二进制补码格式。要求完成设计源文件和仿真源文件的编制, 并进行仿真验证。

答: 逻辑功能表如下:

十进制	二进制原码	二进制补码
0	0000/1000	0000
1	0001	0001

2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
-1	1001	1111
-2	1010	1110
-3	1011	1101
-4	1100	1100
-5	1101	1011
-6	1110	1010
-7	1111	1001

Verilog 代码如下：

```

module exercises3_32 (
    input wire [3:0] sign_mag,    // 输入：符号-幅度格式（最高位为符号位）
    output reg [3:0] twos_comp    // 输出：二进制补码格式
);

always @(*) begin
    if (sign_mag[3] == 1'b0) begin
        twos_comp = sign_mag;    // 正数直接输出
    end else begin
        if (sign_mag[2:0] == 3'b000) begin
            twos_comp = 4'b0000;    // 处理-0 的情况，转换为 0
        end else begin
            // 负数：数值部分取反加 1，保留符号位
            twos_comp = {1'b1, (~sign_mag[2:0] + 3'b001)};
        end
    end
end
end
endmodule

```

仿真代码如下：

```
`timescale 1ns / 1ps
```

```

module tb_exercises3_32 ();
// 输入输出信号声明
reg [3:0] sign_mag;
wire [3:0] twos_comp;

// 实例化被测测试模块
exercises3_32 uut (
    .sign_mag(sign_mag),
    .twos_comp(twos_comp)
);

// 测试过程
initial begin
    // 初始化输入
    sign_mag = 4'b0000; // 初始值
    // 测试用例 1: 正数 (符号位为 0)
    sign_mag = 4'b0000; #10; // +0 → 0000
    sign_mag = 4'b0001; #10; // +1 → 0001
    sign_mag = 4'b0111; #10; // +7 → 0111
    // 测试用例 2: 负数 (符号位为 1)
    sign_mag = 4'b1000; #10; // -0 → 0000
    sign_mag = 4'b1001; #10; // -1 → 1111
    sign_mag = 4'b1011; #10; // -3 → 1101
    sign_mag = 4'b1111; #10; // -7 → 1001
    // 测试用例 3: 边界值
    sign_mag = 4'b0000; #10; // 最小正数
    sign_mag = 4'b1111; #10; // 最大负数 (-7)
    sign_mag = 4'b1001; #10; // 中间负数 (-1)
end
endmodule

```

3.33 利用 Verilog HDL 行为建模方式设计一个逻辑电路，将 4 位二进制数字从格雷码转换为二进制码。要求完成设计源文件和仿真源文件的编制，并进行仿真验证。

答：逻辑功能表如下：

格雷码	二进制原码
0000	0000
0001	0001

0011	0010
0010	0011
0110	0100
0111	0101
0101	0110
0100	0111
1100	1000
1101	1001
1111	1010
1110	1011
1010	1100
1011	1101
1001	1110
1000	1111

Verilog 代码如下：

```

module exercises3_33 (
    input wire [3:0] gray,    // 输入：4 位格雷码
    output wire [3:0] bin     // 输出：4 位二进制码
);

// 格雷码转二进制码逻辑
assign bin[3] = gray[3];           // 最高位相同
assign bin[2] = bin[3] ^ gray[2];  // 次高位为前一位二进制码与当前格雷码异或
assign bin[1] = bin[2] ^ gray[1];
assign bin[0] = bin[1] ^ gray[0];
endmodule

```

仿真代码如下：

```

`timescale 1ns / 1ps
module tb_exercises3_33 ();

// 信号声明
reg [3:0] gray;

```



```

wire [3:0] bin;

// 实例化被测试模块
exercises3_33 uut (
    .gray(gray),
    .bin(bin));

// 测试逻辑
initial begin
    // 初始化输入
    gray = 4'b0000;
    // 遍历所有可能的格雷码输入
    #10 gray = 4'b0000; // 二进制 0000
    #10 gray = 4'b0001; // 二进制 0001
    #10 gray = 4'b0011; // 二进制 0010
    #10 gray = 4'b0010; // 二进制 0011
    #10 gray = 4'b0110; // 二进制 0100
    #10 gray = 4'b0111; // 二进制 0101
    #10 gray = 4'b0101; // 二进制 0110
    #10 gray = 4'b0100; // 二进制 0111
    #10 gray = 4'b1100; // 二进制 1000
    #10 gray = 4'b1101; // 二进制 1001
    #10 gray = 4'b1111; // 二进制 1010
    #10 gray = 4'b1110; // 二进制 1011
    #10 gray = 4'b1010; // 二进制 1100
    #10 gray = 4'b1011; // 二进制 1101
    #10 gray = 4'b1001; // 二进制 1110
    #10 gray = 4'b1000; // 二进制 1111
end
endmodule

```

3.34 利用 Verilog HDL 行为建模方式设计含有一个输入 x 和一个输出 z 的同步时序电路，能够识别输入序列中的“10”码。即只要在两个时钟周期内输入 x 连续出现 1 和 0，则电路输出 $z = 1$ ；否则， $z = 0$ 。要求完成设计源文件和仿真源文件的编制，并进行仿真验证。

答：Verilog 代码如下：

```

module exercises3_34 (
    input clk,        // 时钟信号
    input rst,        // 复位信号

```

```

    input x,          // 输入序列
    output reg z      // 输出信号
);

reg prev_x; // 保存前一个时钟周期的输入值

always @(posedge clk or negedge rst) begin
    if (!rst) begin
        // 复位时清零
        prev_x <= 1'b0;
        z <= 1'b0;
    end
    else begin
        // 更新前一个输入值
        prev_x <= x;
        // 输出逻辑：检测前一个为 1 且当前为 0
        z <= prev_x & ~x;
    end
end
end
endmodule

```

仿真代码如下：

```

`timescale 1ns / 1ps
module tb_exercises3_34 ();

// 定义信号
reg clk;
reg rst;
reg x;
wire z;

// 实例化被测模块
exercises3_34 uut (
    .clk(clk),
    .rst(rst),

```

```

        .x(x),
        .z(z));
// 生成时钟（周期 10ns）
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// 测试逻辑
initial begin
    // 初始化复位
    rst = 0;
    x = 0;
    #20;      // 等待两个完整时钟周期
    rst = 1;   // 复位结束

    // 测试用例 1：输入序列 1→0（应检测成功）
    x = 1;
    #10; // 等待一个时钟周期，prev_x=1，z=0
    x = 0;
    #10; // 此时 z=1

    // 测试用例 2：输入序列 1→1（应检测失败）
    x = 1;
    #10;
    x = 1;
    #10;

    // 测试用例 3：输入序列 0→1（应检测失败）
    x = 0;
    #10;
    x = 1;
    #10;

    // 测试用例 4：连续输入 1→0→1→0（两次检测成功）
    x = 1;
    #10;

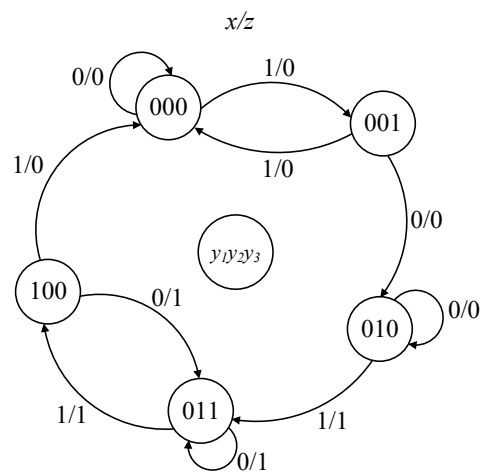
```

```

x = 0;
#10;
x = 1;
#10;
x = 0;
#10;
end
endmodule

```

3.35 图题 3.1 为米利型时序逻辑电路的状态图，状态变量是 $y_1y_2y_3$ ，输入变量为 x ，输出函数为 z ，初始状态为 $y_1y_2y_3 = 000$ 。利用 Verilog HDL 状态机建模方式完成设计源文件和仿真源文件的编制，并进行仿真验证。



图题 3.1 米利型时序逻辑电路状态图

答：Verilog 代码如下：

```

module exercises3_35(
    input    x,
    input    clk,
    input    rst,
    output reg z);

//状态划分
parameter STATE0 = 3'b000;
parameter STATE1 = 3'b001;
parameter STATE2 = 3'b010;
parameter STATE3 = 3'b011;
parameter STATE4 = 3'b100;

```

```

reg [2:0] current_state;      //现态
reg [2:0] next_state;        //次态

//第一个 always 模块： 现态跟随次态，实现同步状态跳转
always@(posedge clk or negedge rst) begin
    if(!rst)
        current_state<= STATE0;    //按下复位键时，当前状态设置为 STATE0
    else
        current_state <= next_state; //将次态赋值给现态
end

//第二个 always 模块： 组合逻辑判断状态转移条件，描述状态转移规律
always@(*) begin
    if(!rst) begin
        next_state= STATE0;
    end
    else begin
        case(current_state)
            STATE0: begin
                if(x == 1'b0)
                    next_state = STATE0;
                else
                    next_state = STATE1;
            end
            STATE1: begin
                if(x == 1'b0)
                    next_state = STATE2;
                else
                    next_state = STATE0;
            end
            STATE2: begin
                if(x == 1'b0)
                    next_state = STATE2;
                else
                    next_state = STATE3;
            end
            STATE3: begin
                if(x == 1'b0)

```

```

        next_state = STATE3;
    else
        next_state = STATE4;
    end
STATE4: begin
    if(x == 1'b0)
        next_state = STATE3;
    else
        next_state = STATE0;
    end
    default: next_state = STATE0;
endcase
end
end

```

//第三个 always 模块：描述状态输出

```
always@(posedge clk or negedge rst) begin
```

```

    if(!rst)
        z = 1'b0;
    else begin
        case(current_state)
            STATE0: z = 1'b0;
            STATE1: z = 1'b0;
            STATE2:
                if(x == 1'b0)
                    z = 1'b0;
                else
                    z = 1'b1;
            STATE3: z = 1'b1;
            STATE4:
                if(x == 1'b0)
                    z = 1'b1;
                else
                    z = 1'b0;
            default: z = 1'b0;
        endcase
    end
end

```

```
end
```

```
endmodule
```

仿真代码如下：

```
`timescale 1ns / 1ps
module tb_exercises3_35();

    reg  x;
    reg  clk;
    reg  rst;
    wire z;

    initial begin
        x = 1'b0;
        clk = 1'b0;
        rst = 1'b0;
        #20
        rst = 1'b1;
        #10
        x = 1'b1;    //跳转到 STATE1, z=0
        #10
        x = 1'b1;    //跳转到 STATE0, z=0
        #10
        x = 1'b1;    //跳转到 STATE1, z=0
        #10
        x = 1'b0;    //跳转到 STATE2, z=0
        #10
        x = 1'b1;    //跳转到 STATE3, z=1
        #10
        x = 1'b1;    //跳转到 STATE4, z=1
        #10
        x = 1'b1;    //跳转到 STATE0, z=0
    end

    always #5 clk = ~clk;

    exercises3_35 uut(
        .x  (x),
        .clk(clk),
        .rst(rst),
        .z  (z));
endmodule
```

```
endmodule
```